FROM OBSERVATION TO AUTONOMY

Bridging an AIOps system from advisory diagnostics to autonomous remediation, without LLM in the execution path

8
phases without LLM in execution path

0
test regressions across 18 sessions

5
consecutive A+ Gemini checkpoints

Michel Mazza · Phase 22-23 · April 2026

AFTER 21 PHASES, THE SYSTEM COULD SEE — BUT NOT ACT

- Diagnoses problems in <10ms across structured event logs and metric snapshots
- Explains its reasoning across 8 Glass Box transparency tabs
- Tells the operator what's wrong — but cannot do anything about it

Phase 22 mission: build the safety architecture before the actor capability — rehearse before executing.

THE NAIVE ANSWER: ADD AN LLM AGENT THAT DECIDES WHAT TO DO

```
LLM → Production State Mutation
```

✗ REJECTED

LLMs hallucinate action classes

Fails by executing the wrong restart on the wrong service

LLMs return malformed plans

Fails mid-execution after partial state mutation

LLMs fail in ways that are hard to enumerate

And harder to test in a production safety context

An LLM in the safety-critical execution path is a liability without commensurate value.

PHASE 22 — SELF-HEALING FOUNDATIONS

Three weeks to teach the system to rehearse action without ever taking it

WEEK 1	WEEK 2	WEEK 3
Test harness floor	**Level 0 — Suggest**	**Level 1 — Simulate**
4/8 Glass Box tabs under regression	YAML allowlist, no LLM in decision path	Three-gate dry-run safety architecture
0% flakiness · 25 new tests	CI green · surplus reinvestment	Kill-switch + rate-limit + loop guard

1036 -> 1108 tests | 0 failures at close (first since Phase 21) | A+ all three checkpoints

PHASE 23 — THE AUTONOMOUS BRIDGE

Three weeks to teach the system to actually act, deterministically

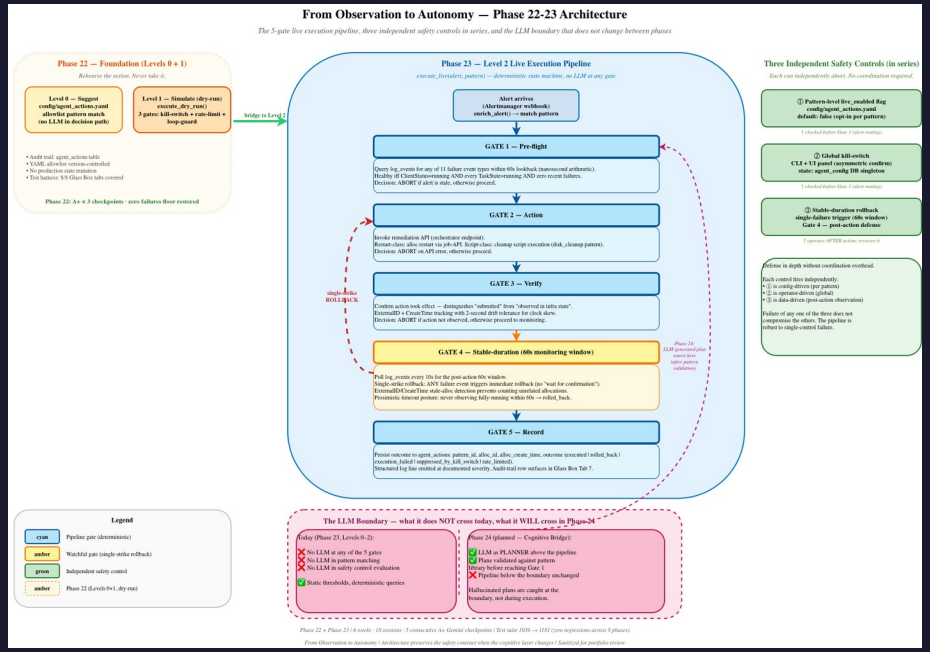
HYPOTHESIS

A deterministic, gate-based execution pipeline with human-in-the-loop overrides can safely bridge Level 1 (advisory) to Level 2 (autonomous remediation) without introducing LLM dependency in the execution path.

✓ Validated by Gemini at Session 0 before any code shipped. Validated by results at Session 9.

8	2	31/31
Consecutive phases zero-LLM in execution	Consecutive A+ Gemini checkpoints	Mandatory tiers shipped

5 GATES · 60-SECOND DECISION WINDOW · SINGLE-STRIKE ROLLBACK



No LLM at any gate. All thresholds are static and configuration-controlled. All checks are deterministic queries.

WHEN THE ACTION MAKES THINGS WORSE, YOU HAVE SECONDS — NOT MINUTES

✗ WRONG

- Wait for 2 or 3 confirmation failures before reverting
- Compounds the original problem
- Operator notices the system 'took its time' deciding

→

✓ RIGHT

- Single failure → immediate rollback
- Contains incident scope to seconds
- ExternalID + CreateTime ensure tracking the same allocation

Err on the side of reverting. We make the cheaper mistake.

A SECOND RESTART-CLASS PATTERN WOULD HAVE PROVED NOTHING NEW

RESTART-CLASS	SCRIPT-CLASS (disk_cleanup)
Existing pattern (Phase 22 dry-run)	New pattern (Phase 23)
Triggers job restart via API	Executes cleanup script
Same action class as previous	Different action class
Easy to ship	Harder to ship — worth it

When extending a system, prefer the extension that proves new capability over the one that proves existing capability twice.

THREE ITERATIONS TO LAND. THE BUG SHIPPED TWICE. THE LESSON HOLDS.

1 Iteration 1

`st.toggle`

State carried forward across reruns — wrong widget

2 Iteration 2

`Sidebarplacement`

Wrong layer fix — misdiagnosed root cause

3 Iteration 3 — Option B

`Move panelto main()`

Root cause addressed — unconditional render on every run

Function tests passed. AppTest passed. Manual smoke test caught it.

THREE-LAYER UI VALIDATION — LESSON ENCODED AS A STANDARD

LAYER	CATCHES	MISSES (by design)
Layer 1 — Function-level	Pure logic bugs, contract violations	Framework state, multi-turn rendering
Layer 2 — AppTest	Single-turn integration, callback wiring	Multi-turn rendering, conditional-render gaps, ergonomics
Layer 3 — Manual smoke	Multi-turn rendering, ergonomics, "feels wrong" issues	Caught at Layer 1+2 with regression suite

All three required. None optional. Codified in Testing-Guide.md v5.4.

BRANCH PROTECTION: WORKING WITH PLATFORM CONSTRAINTS

OK **Configured** branch protection on main: status checks required, branches must be up to date

! **Not enforced** — GitHub does not enforce classic branch protection on private repos under the Free plan

OK **Compensating controls implemented:** CC four-skill rhythm enforces locally what CI would enforce post-merge

OK **Activates automatically** on plan upgrade or repo visibility change

Configure controls that won't currently fire. Document the constraint. Implement compensating controls.

SIX WEEKS · TWO PHASES · ZERO SHORTCUTS

8
Phases without LLM
in execution path

0
Test regressions
across 18 sessions

5
Consecutive A+
Gemini checkpoints

1036 → 1181
Test suite growth (+145)

6 → 8
Zero-LLM streak (phases)

19 → 0
Prometheus skip count

Combined Phase 22-23: 145 new tests · 19 long-standing skips closed · 3 new architectural patterns codified into project standards

PHASE 24 — THE COGNITIVE BRIDGE

PRESERVED (unchanged)

- ✓ 5-gate execution pipeline (unchanged)
- ✓ Three independent safety controls (unchanged)
- ✓ Pattern allowlist + dry-run validation discipline (unchanged)

→

NEW (Phase 24)

LLM as planner — generates action plans

Plans validated against pattern library — before reaching Gate 1

Hallucination caught at pipeline boundary — not during execution

The architectural payoff: the safety contract does not change when the cognitive layer changes.

FROM OBSERVATION TO AUTONOMY

> *We built a deterministic, safe, and verifiable autonomous remediation system that operates with nanosecond precision and human-in-the-loop overrides.*

- External validation, Phase 23 Week 2

8	0	5
phases without LLM in execution path	regressions across 18 sessions	consecutive A+ Gemini checkpoints

The discipline holds.

Questions?